

The Network Bouncer

Detecting Suspicious Port Scanning in Data Center Traffic

1. Problem Title

The Network Bouncer: Detecting Suspicious Port Scanning in Data Center Traffic

2. Background / Context

In a data center, servers constantly communicate with each other to exchange data. This internal communication is called East-West Traffic. Under normal conditions, each server talks to a limited set of known machines (neighbors). However, when a system is compromised (e.g., malware infection), it may start behaving abnormally by attempting to connect to many different machines across multiple ports. This behavior is known as Port Scanning and is often the first step in spreading an attack.

3. Problem Description (The Core Problem)

Given network traffic data (CSV format), identify when a machine behaves suspiciously by:

- Connecting to many different destination IPs
- Trying multiple ports
- Doing so within a short time window

The challenge is to build a system that can:

- analyze network metadata
- detect abnormal patterns
- classify traffic as Normal or Suspicious (Backdoor/Analysis)

4. Objectives / Desired Outcomes

- Data Parsing: Read and process CSV-based network traffic logs
- Behavior Analysis: Track how each source IP communicates
- Anomaly Detection: Identify port scanning patterns
- Classification: Label traffic as:
 - Normal
 - Suspicious (Backdoor / Analysis)
- Reporting: Generate a clear output showing suspicious machines

5. Scope Definition

In-Scope:

- CSV parsing (UNSW-NB15 dataset)
- Feature extraction (src IP, dst IP, ports, protocol, etc.)
- Detection logic (rule-based or simple ML)
- Basic reporting (console / CSV / simple UI)

Out-of-Scope:

- Real-time network monitoring

- Packet-level deep inspection
- Advanced ML models (deep learning, etc.)

Evaluation Focus:

- Problem understanding
- Architecture clarity
- Usability of output
- Design reasoning

6. Constraints / Limitations

- Use standard Python libraries (pandas, collections, etc.)
- Must run on a local machine
- Dataset may be large → avoid inefficient full-memory usage
- Detection logic should be simple and explainable

7. Assumptions

- Each row represents a network connection/event
- Dataset is mostly clean but may contain noise
- Port scanning = high number of unique connections in short time
- Students have basic Python knowledge

8. Target Users / Personas

Security Analyst (SOC Analyst):

- Needs to quickly identify suspicious machines
- Requires simple, interpretable output
- Works under time pressure

9. Current State Summary

Without automated tools:

- Analysts manually inspect logs
- Difficult to detect patterns across thousands of rows
- Port scanning behavior is often missed

10. Expected Target State / Vision

A simple tool where a user can run:

```
python network_bouncer.py network_log.csv
```

And get output like:

```
Suspicious Activity Detected:
```

```
Source IP: 192.168.1.10
Connections: 150
Unique Destinations: 80
```

Unique Ports: 60
Classification: Backdoor/Analysis

11. Success Metrics & Acceptance Criteria

- Detects simulated port scanning behavior correctly
- Processes large CSV files without crashing
- Output clearly includes:
 - Source IP
 - Number of connections
 - Unique IPs / ports
 - Classification
- Handles malformed or missing data gracefully

12. Risks & Dependencies

- Risks:
 - High-traffic systems may appear suspicious (false positives)
 - Dataset imbalance (more normal traffic than attacks)
- Mitigation:
 - Use configurable thresholds
 - Allow tuning of sensitivity

13. Expected Deliverables from Participants

- Python Script: network_bouncer.py
- Output Report (CSV or console output)
- Screenshot of results
- Short explanation of approach

14. Provided Inputs / Resources for Hackathon Teams

- Dataset: UNSW-NB15 (Lite CSV)
 - <https://www.kaggle.com/datasets/mrwellsdavid/unswnb15>
- Key Columns:
 - srcip → Source IP
 - dstip → Destination IP
 - sport → Source Port
 - dsport → Destination Port
 - proto → Protocol
 - service → Service
 - state → Connection state
 - label → (optional, for validation)

15. Evaluation Criteria

Section	Max	What is Expected
Problem Understanding	10	Demonstrates clear understanding of port scanning behavior, risks, and possible false positives
User Thinking	10	Designs output to help analysts quickly identify and act on suspicious activity
Architecture	20	Clean modular design (parsing → analysis → detection → reporting)
Logic	15	Correct implementation of detection logic (IP/port analysis, time patterns)
Resilience	10	Handles bad or missing data without crashing
Reporting	10	Clear and actionable output with relevant metrics
Scalability	10	Efficient processing of large datasets
Tradeoffs	5	Explains design decisions (e.g., rule-based vs ML)
Explainability	5	Clearly explains system flow and logic
Adaptation	5	Can extend solution to new formats or attack types
Total	100	

16. Stretch Goals (Bonus Points)

- Add **time-window-based detection**
- Visualize suspicious activity (charts)
- Implement simple ML classification
- Add severity levels (Low / Medium / High)
- Detect multiple attack patterns

17. Additional References / Links

Dataset & Documentation

- <https://www.kaggle.com/datasets/mrwellsdavid/unswnb15>
- <https://research.unsw.edu.au/projects/unswnb15-dataset>

Cybersecurity Concepts

- <https://www.cloudflare.com/learning/security/what-is-port-scanning/>

- <https://www.sans.org/white-papers/intrusion-detection-systems/>

Technical References

- <https://pandas.pydata.org/docs/>
- <https://docs.python.org/3/library/collections.html>

Optional Learning

- <https://developers.google.com/machine-learning/anomaly-detection>
- <https://www.cisco.com/c/en/us/products/security/what-is-network-traffic-analysis.html>